

OPC UA

Industrial Communication Protocol Study Guide

For SCADA Automation Engineers
Secure Channel · Address Space · Subscriptions · Certificates

Ricardo Barbosa
NERC-Certified EDF Operator
MS Artificial Intelligence, Johns Hopkins University

May 2026

Contents

Contents	1
1 What Is OPC UA?	3
1.1 Why OPC Classic Failed	3
1.2 What OPC UA Fixed	4
1.3 OPC UA in the Wild	4
2 OPC UA Architecture	5
2.1 OPC UA Client-Server Diagram	5
2.2 Secure Channel Lifecycle	5
2.3 Deployment Patterns	6
3 Information Model & Address Space	7
3.1 NodeId Formats	7
3.2 Node Classes	8
3.3 References	8
4 OPC UA Services	9
4.1 Key Services	9
4.2 Read vs. Subscribe	10
5 Security & Certificates	11
5.1 SecurityMode Comparison	11
5.2 Certificate Trust Workflow	11
5.3 Security Policies	12
6 Subscriptions & MonitoredItems	13
6.1 Key Parameters	13
6.2 Deadband Filtering	14
7 Transport & Encoding	15
7.1 Transport Comparison	15
7.2 Binary Encoding Built-In Types	15

8	OPC UA in Ignition & RTAC	17
8.1	Ignition OPC UA Client Workflow	17
8.2	SEL RTAC as OPC UA Server	17
9	Troubleshooting OPC UA	19
9.1	Common Status Codes	19
9.2	UA Expert Diagnostic Workflow	20
10	Lab & Tools	21
10.1	Required Tools (All Free)	21
10.2	Lab Exercises	21

Chapter 1

What Is OPC UA?

OPC UA — Unified Architecture — is an industrial communication standard published by the OPC Foundation in 2008. It defines how machines, sensors, PLCs, SCADA systems, and enterprise software exchange data in a platform-independent, secure, and structured way.

Key Concept

OPC UA is a platform-independent, secure, structured data exchange protocol for industrial automation. It replaced OPC Classic (COM/DCOM-based) with a system that runs on Windows, Linux, and embedded hardware without requiring Windows registry configuration or DCOM firewall exceptions.

1.1 Why OPC Classic Failed

OPC Classic (OPC-DA) used Microsoft's COM/DCOM technology. This required:

- Windows-only deployment — no Linux, no embedded systems
- COM/DCOM registration on both client and server machines
- DCOM firewall configuration that varied by Windows version
- No built-in security — no encryption, no certificate authentication

WARNING

OPC Classic servers still run in production facilities built before 2010. Many Ignition installations connect to legacy OPC-DA servers via the OPC-COM module. Knowing OPC UA does not mean you will never encounter OPC-DA.

1.2 What OPC UA Fixed

Feature	Description
Platform Independence	No COM/DCOM. Runs on Windows, Linux, embedded systems.
Built-In Security	Certificate authentication, session encryption, message signing.
Information Model	Browsable address space of typed nodes — not a flat tag list.
Transport Flexibility	Binary TCP (fastest), HTTPS (IT-friendly), WebSocket.
Companion Specs	Domain-specific extensions for PLCopen, PackML, ISA-95, etc.

1.3 OPC UA in the Wild

OPC UA is deployed in: Inductive Automation Ignition (built-in server on port 62541), Siemens S7-1200/1500 (native OPC UA in TIA Portal V14+), SEL RTAC (built-in OPC UA server), Beckhoff TwinCAT 3, Rockwell FactoryTalk Linx, and all major DCS vendors (ABB, Emerson, Honeywell).

The specification spans 14 parts. Part 1 (Overview), Part 3 (Address Space), Part 4 (Services), and Part 6 (Mappings) are the essential reading.

Chapter 2

OPC UA Architecture

OPC UA uses a client-server model. The server exposes an Address Space. The client connects, establishes a Secure Channel, opens a Session, then browses, reads, writes, and subscribes to data.

2.1 OPC UA Client-Server Diagram

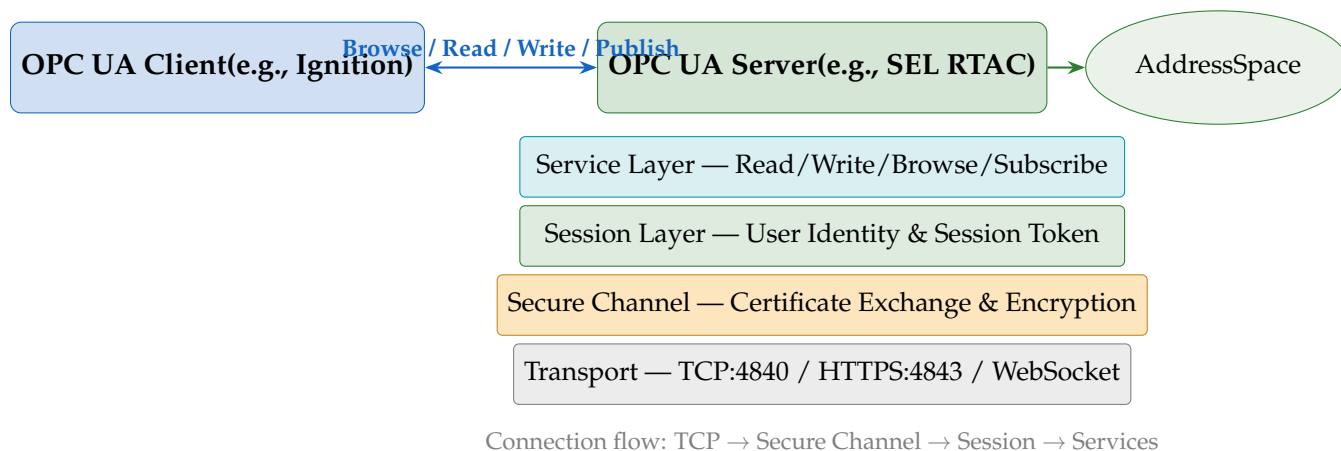


Figure 2.1: OPC UA Client-Server Architecture with Layered Security Model

2.2 Secure Channel Lifecycle

Listing 2.1: OPC UA Connection Sequence

```
CLIENT -> TCP Connect to server:4840
```

```
CLIENT -> OpenSecureChannel(SecurityMode, SecurityPolicy, ClientCert)
SERVER <- OpenSecureChannelResponse(ServerCert, SessionKeys)
CLIENT -> CreateSession(ClientCert, Endpoint)
SERVER <- CreateSessionResponse(SessionId, AuthToken)
CLIENT -> ActivateSession(UserIdentityToken)
SERVER <- ActivateSessionResponse(OK)
CLIENT -> Read / Browse / Subscribe [all within Session]
```

Field Gotcha

Certificate errors occur at `OpenSecureChannel` — before any Session is created. If you see `BadCertificateUntrusted`, the server received the client certificate but hasn't trusted it. Fix: add the client cert to the server's trusted store, and add the server cert to the client's trusted store. OPC UA is mutual authentication — both sides must explicitly trust each other.

2.3 Deployment Patterns

Embedded Server OPC UA runs directly on the device (Siemens S7-1500, SEL RTAC, Beckhoff TwinCAT).

Gateway Server Middleware connecting field devices and re-exposing them as OPC UA (Ignition, KEPServerEX).

Aggregation Server Merges multiple OPC UA servers into one Address Space.

Historical Access Server Exposes historian data via OPC UA Part 11 (Osisoft PI, etc.).

Chapter 3

Information Model & Address Space

The OPC UA Address Space is a browsable, typed graph of nodes. Every data point, every object, and every method is a node. Nodes are connected by typed References.

3.1 NodeId Formats

Every node has a NodeId — a unique identifier within a Namespace. Four formats exist:

NodeId Format Examples

Format	Example
Numeric	ns=2;i=1001
String	ns=2;s=Temperature_PV
GUID	ns=2;g=550e8400-e29b-41d4-a716-446655440000
Opaque	ns=2;b=AAEC

Namespace 0 (ns=0) is reserved for OPC UA specification nodes. Custom nodes use ns=1 or higher. Numeric NodeIds are most common in auto-generated Address Spaces. String NodeIds are typical in human-readable custom implementations.

3.2 Node Classes

Node Class	Purpose	Example
Object	Container — represents physical or logical entity	PLC, pump, device
Variable	Holds a typed value. Primary data node.	Temperature, speed
Method	Callable server-side function	StartPump()
ObjectType	Type definition for Objects (like a class)	PumpType
VariableType	Type with value constraints	AnalogItemType
ReferenceType	Semantic meaning of a node relationship	HasComponent
DataType	Data type for Variable values	Float, Structure
View	Pre-defined Address Space subset	Diagnostic view

3.3 References

Key reference types:

- HasComponent — Object HasComponent Variable/Method/Object
- HasProperty — Descriptive attributes (engineering units, range)
- Organizes — Folder-style grouping
- HasTypeDefinition — Every Object/Variable has exactly one type
- HasSubtype — Type inheritance

Key Concept

Every Variable carries three things: a **Value**, a **StatusCode** (Good/Uncertain/Bad), and a **SourceTimestamp**. A Bad StatusCode means the value is not trustworthy. SCADA systems must not display Bad-status values as valid process data.

Chapter 4

OPC UA Services

OPC UA defines 37 services in 11 service sets. All client-server interaction is a service call — a request/response pair with defined parameters and StatusCodes.

4.1 Key Services

Service Set	Key Services	Use Case
Discovery	FindServers, GetEndpoints	Server discovery
SecureChannel	OpenSecureChannel	Transport security
Session	CreateSession, ActivateSession	Connection management
View	Browse, TranslateBrowsePaths	Navigate Address Space
Attribute	Read, Write	Data access
Method	Call	Invoke server functions
MonitoredItem	CreateMonitoredItems	Set up monitoring
Subscription	CreateSubscription, Publish	Push data delivery

4.2 Read vs. Subscribe

Read vs Subscribe Comparison

Read (Polling)	Subscribe (Event-Driven)
Client requests value every N ms	Server monitors and notifies on change
Network traffic $\propto$ poll rate $\times$ tags	Network traffic $\propto$ change rate
Always gets current value	Gets value when it changes
<i>Use for: one-time reads, commissioning</i>	<i>Use for: live SCADA, historian feed</i>
500 tags @ 500ms = 1000 req/sec	500 tags, 3 changing = 3 notifications

Field Gotcha

Ignition uses subscriptions, not polling, for OPC UA tags. When you create an OPC UA tag in Ignition, it creates a MonitoredItem in a Subscription. This is why Ignition is more network-efficient than Modbus TCP polling for large tag counts.

Chapter 5

Security & Certificates

OPC UA has well-designed security: mutual certificate authentication, encrypted sessions, signed messages. The practical problem: 90% of OPC UA deployments use `SecurityMode=None` because certificate configuration is hard and the deadline was yesterday.

5.1 SecurityMode Comparison

SecurityMode Comparison				
		None	Sign	Sign&Encrypt
Message Signing		×		
Encryption		×	×	
Certificates	Re-	No	Yes (both)	Yes (both)
quired				
Traffic Readable	Yes		Yes	No
Suitable for Pro-	No		Marginal	Yes
duction				

5.2 Certificate Trust Workflow

1. Server generates self-signed X.509 certificate on first startup
2. Client connects, receives server certificate
3. Client checks its trusted store — if not found: `BadCertificateUntrusted`
4. Client also presents its certificate (mutual auth)

5. Server checks its trusted store — if not found: connection rejected
6. Both sides must explicitly trust each other's certificates

WARNING

SecurityMode=None means all OPC UA traffic is unencrypted and unsigned. Any device on the control network can read process data. On a network with any external connectivity, this is a critical vulnerability.

5.3 Security Policies

Use **Basic256Sha256** as the minimum acceptable security policy. Basic128Rsa15 and Basic256 use SHA-1 (broken) and are deprecated. The Aes256_Sha256_RsaPss policy provides the highest currently available security.

Chapter 6

Subscriptions & MonitoredItems

A Subscription is a container on the server. MonitoredItems are the data points being watched inside it. The server samples MonitoredItems at the SamplingInterval and sends notifications to the client at the PublishingInterval.

6.1 Key Parameters

Parameter	Description	Typical
PublishingInterval	Rate at which server delivers notifications (ms)	500–5000 ms
LifetimeCount	Sessions before subscription deleted without Publish	3×MaxKeepAlive
MaxKeepAliveCount	Empty intervals before KeepAlive sent	10
SamplingInterval	Per-item: how fast server samples source (ms)	100–1000 ms
QueueSize	Per-item: notification buffer depth	1–10

Key Concept

SamplingInterval can be faster than PublishingInterval. The server samples data into a queue at SamplingInterval and delivers the accumulated notifications at PublishingInterval. QueueSize controls how many samples are buffered between deliveries.

6.2 Deadband Filtering

Absolute Deadband Suppress if $|\text{new} - \text{old}| < \text{deadband}$. Example: only notify if temperature changes by more than 0.5°C .

Percentage Deadband Suppress if $\text{change} < X\%$ of the EURange. Requires EURange property on the Variable.

Chapter 7

Transport & Encoding

OPC UA separates the data model from the transport. Three transport mappings are defined.

7.1 Transport Comparison

Transport	Port	Encoding	Notes
UA-TCP Binary	4840	Binary	Fastest, most common, use this
HTTPS	4843	Binary or JSON	For IT/OT integration, proxies
WebSocket	4840/443	Binary or JSON	Browser clients, rarely needed

7.2 Binary Encoding Built-In Types

OPC UA defines 25 built-in types. Key ones: Boolean (1 byte), Int32 (4), Float (4), Double (8), String (4+N), DateTime (8 bytes, 100-nanosecond ticks since 1601-01-01), StatusCode (4), NodeId (variable), Variant (type+value), DataValue (value+status+timestamps), ExtensionObject (any type).

Pro Tip

The Variant type can hold any built-in type plus arrays. Variable values are transmitted as Variants. This allows OPC UA's heterogeneous Address Space to use a single serialization format for all data types.

Chapter 8

OPC UA in Ignition & RTAC

Ignition is simultaneously an OPC UA server (exposes tags to external clients on port 62541) and an OPC UA client (connects to PLCs and RTACs to read process data).

8.1 Ignition OPC UA Client Workflow

1. Gateway → OPC UA → Connections → Add connection
2. Enter server endpoint URL (`opc.tcp://192.168.1.100:4840`)
3. First connect attempt downloads server certificate to Quarantine folder
4. Trust the server certificate: Gateway → OPC UA → Certificates → Quarantined → Trust
5. Export Ignition's client certificate and import it into the server's trusted store
6. Connection status should change to Connected

Field Gotcha

The most common commissioning mistake: trusting the server certificate in Ignition but forgetting that the server also needs to trust Ignition's certificate. OPC UA is mutual authentication. Both sides trust both certificates. If you've done only one side, the connection fails from the server's perspective.

8.2 SEL RTAC as OPC UA Server

The SEL RTAC includes a built-in OPC UA server (AcSELeator RTAC → Communications → OPC UA Server). It generates a self-signed certificate. The RTAC must

Ricardo Barbosa

trust Ignition's client certificate; Ignition must trust the RTAC's server certificate. Both certificates are self-signed — the PKI trust is entirely manual.

Chapter 9

Troubleshooting OPC UA

OPC UA provides specific status codes for every failure mode. The diagnostic ladder: ping → port check → service running → UA Expert with SecurityMode=None → specific error code.

9.1 Common Status Codes

Status Code	Cause and Fix	When
BadCertificateUntrusted	Certificate not in trusted store. Trust on both sides.	OpenSecureChannel
BadSecureChannelClosed	Channel closed by server (restart, timeout, cert expiry). Reconnect.	During session
BadNodeIdUnknown	NodeId doesn't exist. Verify via Browse. Check namespace index.	Read/Write
BadUserAccessDenied	User lacks permission or wrong credentials.	ActivateSession
BadSubscriptionIdInvalid	Subscription deleted (session loss). Recreate subscription.	Publish
BadTooManySubscriptions	Server subscription limit reached. Delete unused subscriptions.	CreateSubscription

9.2 UA Expert Diagnostic Workflow

1. Connect with SecurityMode=None — if this works, it's a security/certificate issue
2. Browse the Address Space — verify NodeIds before writing client code
3. Subscribe to nodes in Data Access View — verify data is changing
4. Check Connection Log for exact error codes
5. Certificate dialog for trust management

Chapter 10

Lab & Tools

10.1 Required Tools (All Free)

Tool	Provider	Use
UA Expert	Unified Automation	OPC UA diagnostic client — the Wireshark of OPC UA
Prosys Simulation Server	Prosys OPC Ltd	Full OPC UA server with simulated process data
Node-RED OPC UA	npm / flows.nodered.org	Flow-based OPC UA client, no coding required
opcua-asyncio (Python)	PyPI (pip install asyncua)	Full Python OPC UA client and server library

10.2 Lab Exercises

1. **Basic Connection:** Install Prosys + UA Expert. Connect with SecurityMode=None. Browse address space.
2. **Subscribe:** Add sinusoidal nodes to Data Access View. Change sampling intervals. Observe behavior.
3. **Security:** Configure Prosys for SignAndEncrypt. Complete the mutual certificate trust workflow.
4. **Python Client:** Write a script using opcua-asyncio. Connect, read, subscribe, write.

5. **Ignition + Prosys:** Connect Ignition to Prosys. Complete the full certificate trust workflow on both sides.

Pro Tip

Complete Lab Exercise 5 before commissioning any real OPC UA device in Ignition. The certificate trust workflow is identical regardless of whether the server is Prosys or an SEL RTAC. Practicing in the lab means commissioning takes 10 minutes instead of an hour.